Code Formatting and Linter Suppressions

An empirical pilot with controlled inputs and historical repository files

Research report | 3 November 2026 | AI-assisted draft for independent review

Abstract

Can Python formatting change warning suppression while preserving executable syntax? We compare Black and Ruff formatting using the same Ruff lint rules and an AST-based correspondence check. Among 200 controlled inputs per formatter, Black produced 68 confirmed suppression changes and Ruff 35; 16 inputs per formatter remained ambiguous. On 184 commonly resolved inputs, the paired difference was 17.93 percentage points. A separate convenience sample of 40 historical repository files yielded no confirmed changes and conservative AST exclusions. These findings establish behavior on the tested corpus, not ecosystem prevalence or general formatter superiority. The artifact retains source snapshots, raw diagnostics, commands, matching evidence and exploratory uncertainty analyses. Detector error rates remain unmeasured because independent held-out labels are unavailable.

Introduction

Comments such as `# noqa: F821` affect static analysis even though ordinary comments do not appear in Python's executable AST. Reflowing a statement can change the relationship between a directive and warnings. A syntactic preservation check therefore does not establish suppression preservation. Conversely, changed line numbers or equal warning counts do not establish a changed directive scope.

This study isolates formatting from ordinary code evolution. Each formatter receives identical original source. The same linter checks original and formatted snapshots with directives both respected and ignored. We compare the visibility of corresponding underlying diagnostics only after validating correspondence. Uncertain cases are retained explicitly. The contribution is a reproducible, narrowly scoped measurement artifact; we make no claim to discovering the general interaction.

Related work

Hu et al. [1] study suppression behavior and evolution; their discussion of line splitting illustrates why warning-count differences require caution. Li and Yang [2] investigate warning tracking across revisions, motivating explicit correspondence rather than coordinate equality. McKeeman [3] describes differential testing: disagreement identifies candidates requiring interpretation. Here the compared property is warning visibility, not formatted text equality. Black's FAQ [4] acknowledges comment movement, and Ruff's pinned formatter documentation [5] describes a list-collapse example that broadens `noqa` coverage. Our controlled experiment measures this known class of interaction under fixed conditions. The literature review is targeted, not systematic; it cannot establish novelty or comprehensive prior-art coverage.

Research questions

- RQ1: Which tested inputs preserve or change suppression?
- RQ2: Do changes hide or expose corresponding warnings?
- RQ3: How do formatters differ on identical inputs?
- RQ4: Are changes observed in separate natural-source files?
- RQ5 (extension): Can detection achieve useful independently measured error rates?

Methodology

Toolchain and observable property

The experiment used CPython 3.11.9, Black 25.1.0 and Ruff 0.11.13 on Windows. These are pinned historical baselines. Formatting and primary linting use explicit isolated settings, line length 88 and target Python 3.11, without preview or fixes. Only F401 (unused import), F821 (undefined name) and F841 (unused local variable) are studied. We never execute the analyzed programs. Ruff serves as both the common linter and one of the formatters; conclusions concern this tool pairing [6].

Pass	Snapshot	Directive handling
A	Original	Respected: visible warnings
B	Original	Ignored: underlying warnings
C	Formatted	Respected: visible warnings
D	Formatted	Ignored: underlying warnings

Within each snapshot, visible diagnostics must be a multiset subset of underlying diagnostics, matched by rule, message and exact range. Across snapshots, the oracle uses rule, AST occurrence and token context rather than coordinates. It requires unique underlying correspondence before attributing visibility changes. A newly hidden warning becomes suppressed; a newly exposed warning becomes visible. Mixed changes, diagnostic changes, AST changes, ambiguity and execution failures have separate outcomes. Preserved includes text-unchanged cases and means preservation only for the selected observable property.

AST dumps parse with type comments enabled and omit ordinary location attributes. Equality is a conservative syntactic gate, not proof of runtime equivalence. Type-ignore line fields can remain inside these dumps. Original and formatted sources, AST dumps, subprocess records, all lint output, matching details and hashes are retained.

Separate datasets and validation

The controlled suite contains 25 AI-assisted templates crossed with 8 directive modes, yielding 200 sources. Repeated executable structures across directive treatments are intentional; the corpus is not a random sample. Modes include no directive, specific or blanket directives at different positions, and file-level forms. The protocol and candidate design were fixed before the main measurement. Development controls and representative comment-neutralization checks support implementation validity, but neither constitutes an independent accuracy evaluation.

Natural inputs come from immutable historical commits of Requests, Flask and pytest, chosen for feasibility rather than probability sampling. Eligible UTF-8 Python files parse under the study grammar, contain a genuine `noqa` comment, meet the size cap and pass documented vendored/generated exclusions. Files are selected by seeded hash rank within repositories, without outcome filtering or replacement with positives. Archived configuration and licenses are retained. Standardized primary lint uses an artificial filename; a separate sensitivity uses actual relative paths and available native Ruff configuration. Neither recreates full project CI, and running Ruff here does not establish upstream Ruff adoption.

Analysis plan and uncertainty

The main unit is a source-formatter comparison, paired by original input. Related treatments are not independent observations. Detailed statistical procedures were specified after earlier outcome counts were known: all interval analyses are exploratory. We report fixed-corpus counts, complete-pair contrasts, unresolved-outcome bounds and conditional cluster resampling. Bootstrap draws resample whole paired families or syntax categories, preserving treatment dependence within clusters [7]. Exchangeability and independence between clusters are hypothetical, not established by the design. No ecosystem inference or significance claim follows.

Results: controlled inputs

Suppression outcomes and paired comparison

Primary outcome	Black	Ruff
Preserved	116	149
Newly hidden	50	14
Newly exposed	18	21
Ambiguous	16	16
Scheduled total	200	200

Mixed, Diagnostic changed, AST changed and Execution failure each had zero controlled cases. Confirmed drift is Newly hidden plus Newly exposed: 68/200 (34.0%) for Black and 35/200 (17.5%) for Ruff. Among resolved cases the corresponding rates are 36.96% and 19.02%. These denominators answer different questions; unresolved cases are never silently counted as preservation.

Paired drift status	Ruff no drift	Ruff drift
Black no drift	116	0
Black drift	33	35

The table covers 184 commonly resolved inputs. The complete-pair Black-minus-Ruff difference is 17.93 percentage points. Every Ruff drift case also has Black drift in this corpus, but drift directions can differ; this is a binary-event table, not identity of formatter behavior. The same 16 inputs are ambiguous for both formatters, concentrated in f-string and exception-binding boundary families.

Exploratory sensitivity and structural interpretation

Paired cluster model	Clusters	Draws	95% difference interval (pp)
Template families	23	20000	8.70 to 28.26
Syntax categories	12	20000	5.92 to 32.14

These are percentile bootstrap model-sensitivity intervals conditional on hypothetical independent exchangeable clusters, not population confidence statements. Coarser grouping changes the dependence assumption; it cannot remove all shared design effects. Separate family-incidence Wilson outputs [8] are retained in the analysis appendix and concern a different event: any drift within a wholly resolved family.

Holding all scheduled inputs fixed and assigning unresolved outcomes adversarially gives drift bounds of 34.0-42.0% for Black and 17.5-25.5% for Ruff. The paired all-scheduled difference is bounded by 8.5-24.5 pp. These are finite-corpus bounds, not confidence intervals, and use a different estimand from the complete-pair model.

Formatting changed text in 133 Black and 118 Ruff comparisons; 57 and 75 respectively still preserved suppression. Thus textual change is an inadequate proxy. The saved structural and treatment tables describe concentration around statement reflow and directive placement without turning deliberately repeated variants into independent discoveries. Standalone figures accompany the report.

Results: natural-source validation

Sampling frame and primary outcomes

Repository and tag	Archive Python files	Eligible	Sampled
Requests v2.32.3	36	7	7
Flask 3.1.0	83	13	13
pytest 8.3.5	256	28	20

Full commit identifiers, URLs, archive hashes, exclusions and source paths are in `datasets/real_world/phase4/manifest.json`. These historical versions and chosen projects do not represent current project behavior or the Python ecosystem.

Repository	Formatter	Preserved	AST excluded	Confirmed drift
Requests	Black	7	0	0
Requests	Ruff	7	0	0
Flask	Black	12	1	0
Flask	Ruff	13	0	0
pytest	Black	18	2	0
pytest	Ruff	19	1	0

There were no natural-source Ambiguous, Diagnostic changed or Execution failure outcomes. All AST exclusions arose solely from shifted `TypeIgnore.lineno` fields; secondary normalization of those fields restored AST equality. Primary outcomes remain excluded, preserving the original rule. This sensitivity concerns representation of type-ignore metadata and does not establish runtime semantics.

Relevance, sensitivity and interpretation

The natural experiment did not reproduce the controlled suppression changes. Many original files lacked any underlying diagnostic from the selected rule set, and most formatted outputs were unchanged. These observations limit the opportunity to observe drift; they do not justify treating the negative result as a positive finding or resampling for favorable examples.

Native-configuration sensitivity completed 26 Flask and 40 pytest comparisons, all with the same primary outcome classifications. Requests produced 14 missing-configuration status records, not completed native measurements. Native full output is retained, but classification still uses only the selected rules. Project configuration sensitivity is not a complete CI reproduction.

Finite eligible-archive bounds account for unsampled files and unresolved sampled outcomes. Requests bounds are 0.00-0.00% for each formatter. Flask bounds are 0.00-7.69% for Black and 0.00-0.00% for Ruff. pytest bounds are 0.00-35.71% and 0.00-32.14%, respectively. These are bounds for the explicitly eligible historical frames, not confidence intervals or rates among all project files. We do not pool repositories into an IID sample.

Discussion

RQ1-RQ3 receive a bounded empirical answer: formatting can change suppression of corresponding selected warnings while passing the study's AST gate. Both hiding and exposure occur. The formatter contrast describes this deliberately constructed corpus and fixed linter configuration; it is not a ranking of formatter quality. Directive handling trades off several layout goals, so a visibility change alone does not identify a software defect or an upstream policy violation.

RQ4 remains negative in the sampled natural files. Controlled examples demonstrate possibility, while the separate archive sample provides limited evidence about occurrence under its own eligibility rules. Their different selection mechanisms and diagnostic relevance prevent direct prevalence comparison. RQ5 remains an extension: matching is implemented and exercised, but useful false-positive and false-negative rates require independent held-out adjudication. We report no precision, recall or F1.

Threats to validity

Construct validity depends on Ruff's selected warning behavior, `--ignore-noqa` semantics, within-snapshot subset checks and the correctness of AST/token anchors. Unsupported or nonunique correspondence remains ambiguous. Equal AST dumps do not prove universal semantic equivalence, and the natural type-ignore exclusions expose limits of that representation. Ordinary comment changes matter to external tools even when executable syntax is stable.

Internal validity is supported by saved commands, raw outputs, hashes, repeat runs and targeted intervention checks. These do not replace independent review. The same AI-assisted process authored implementation, fixtures, tests and interpretation, creating correlated error risk. Development controls are not an independent test set. Natural repeats were text-unchanged and therefore provide limited evidence about repeating changed natural inputs. Failures, exclusions and historical protocol amendments remain inspectable.

External validity is limited by pinned versions, one operating system, a small selected rule set, authored structures, convenience repositories, historical tags and UTF-8/grammar/size restrictions. Many natural files are irrelevant to the selected warnings. The standardized artificial filename and partial native configuration differ from full project execution. Upstream AI use is unknown. Claims about contemporary releases, other linters, other languages or ecosystem frequency are unsupported.

Conclusion validity is constrained by related template treatments and post-count statistical choices. Conditional cluster intervals and family-level Wilson calculations are sensitivity models, not evidence of random sampling. Bounds and intervals concern different estimands. No significance, general superiority, originality priority or publication-readiness claim is warranted.

Future research

Independent reviewers should adjudicate a held-out set of changed and unchanged pairs, including difficult matching boundaries, before estimating detector accuracy. Further work could sample broader repositories prospectively, study warning-relevant natural files as a separately defined stratum, compare tool versions and configurations, and investigate more diagnostic families. Any AST-gate revision should be prospectively specified and evaluated separately. A larger study should define the population and sampling design before inferential analysis.

Conclusion and AI-use disclosure

This pilot provides reproducible evidence of controlled formatter-associated suppression changes and a separate negative natural-source result. Its strongest contribution is an inspectable measurement procedure with explicit uncertainty and scope. Codex assisted with research engineering, controlled candidate authorship, testing, analysis, literature checking and report drafting. Natural inputs were archived without study-generated edits; their authors' AI practices are unknown. Human scientific review, independent labels and authorship verification remain necessary before submission or external claims. See `docs/ai_use.md` for the detailed disclosure.

References

[1] Huimin Hu, Yingying Wang, Julia Rubin, and Michael Pradel. 2025. An Empirical Study of Suppressed Static Analysis Warnings. Proceedings of the ACM on Software Engineering 2 (FSE), 290-311. DOI: 10.1145/3715729. Published author PDF. The supplied UBC draft has provisional publication metadata; final metadata was checked against the author PDF and Crossref.

[2] Junjie Li and Jinqiu Yang. 2024. Tracking the Evolution of Static Code Warnings: The State-of-the-Art and a Better Approach. IEEE Transactions on Software Engineering 50(3), 534-550. DOI: 10.1109/TSE.2024.3358283. Author PDF.

[3] William M. McKeeman. 1998. Differential Testing for Software. Digital Technical Journal 10(1), 100-107. Original journal PDF, PDF pages 103-110. Used as methodological context, not a claim that disagreement proves a bug.

[4] Black contributors. Black FAQ, pinned version 25.1.0. Versioned source. Current code style and current FAQ were also consulted; current documentation is not evidence of the installed version's exact behavior.

[5] Astral contributors. Known Deviations from Black, Ruff 0.11.13. Versioned source. Current documentation. Documentation establishes prior awareness of the interaction; the retained invocations establish this study's outcomes.

[6] Astral contributors. Ruff linter documentation and Ruff formatter documentation. Accessed 3 November 2026. Current pages provide context; pinned executable help and actual saved commands govern reproduction.

[7] StataCorp. Bootstrap resampling manual, especially cluster resampling. Primary manual. The study implements paired cluster resampling in NumPy; it does not use Stata.

[8] NIST/SEMATECH. e-Handbook of Statistical Methods, confidence limits for a proportion. Wilson method guidance. This supports the formula, not independence of the study's families.

Data availability and reproduction

The local repository contains `datasets/controlled/phase3`, `datasets/real_world/phase4`, raw trial evidence under `datasets/results`, and `reports/phase5/analysis/metrics.json` with its companion tables. Repository URLs, commits, provenance, configurations and retained licenses accompany natural sources. No hosted public release is asserted.

Use Python 3.11 and `requirements-analysis-lock.txt` for study reproduction, following `docs/reproducibility.md`. Existing evidence directories reject overwrites. Regenerate this Markdown report with `python scripts/build_report.py`; verify claims and source hashes using `python scripts/build_report.py --check`. `report/claim_spec.json` identifies evidence fields; `reports/phase6/claim_audit.json` records derived values and template locations. The audit detects stale numeric bindings, not incorrect prose interpretation or errors already present in source evidence.

Optional PDF rendering uses a separate environment with `requirements-report.txt` and `python scripts/render_report.py`. Rendering does not rerun experiments. The PDF and Markdown share content; page breaks are presentation markers. Standalone analysis figures are linked below rather than compressed into the report.

- Outcome distributions
- Paired comparison and model sensitivity
- Structural patterns
- Natural-source diagnostic relevance